

Report - Usare Wireshark per Osservare l'Handshake a 3 Vie TCP

S11L2 - Cisco CyberOps, Mininet, tcpdump, Wireshark

1. Obiettivo

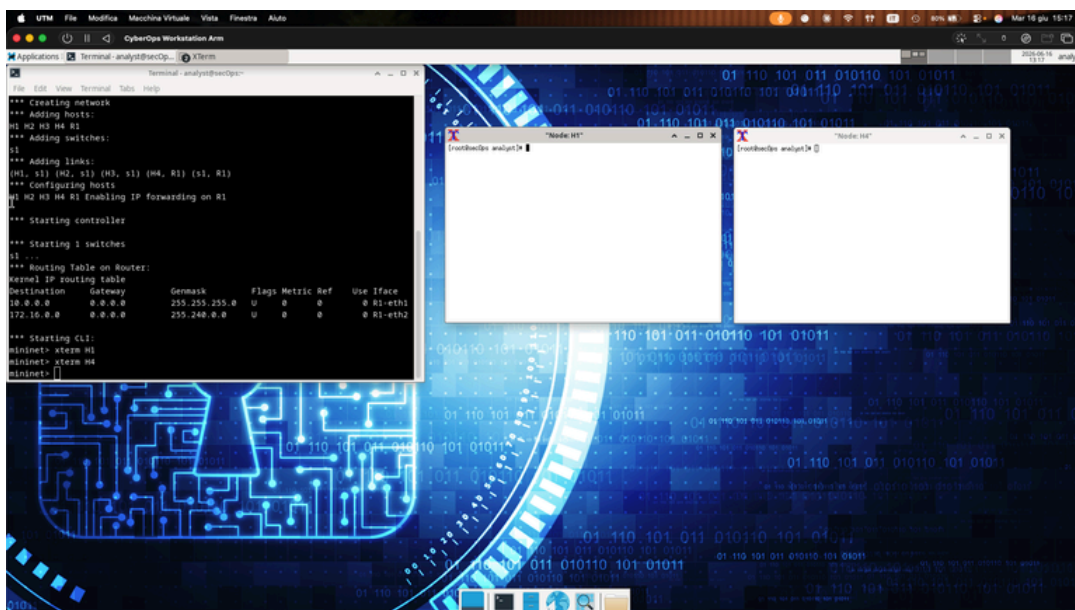
L'obiettivo del laboratorio S11L2 è osservare in tempo reale come funziona il **TCP Three-Way Handshake**, cioè la procedura di stabilimento di una connessione TCP, utilizzando gli strumenti **tcpdump** e **Wireshark** su una topologia di rete emulata con **Mininet**. Il laboratorio si articola in tre parti: configurazione della topologia e cattura dei pacchetti, analisi in Wireshark dei tre frame che compongono l'handshake (SYN, SYN-ACK, ACK), e infine utilizzo di tcpdump per rileggere il file di cattura da riga di comando.

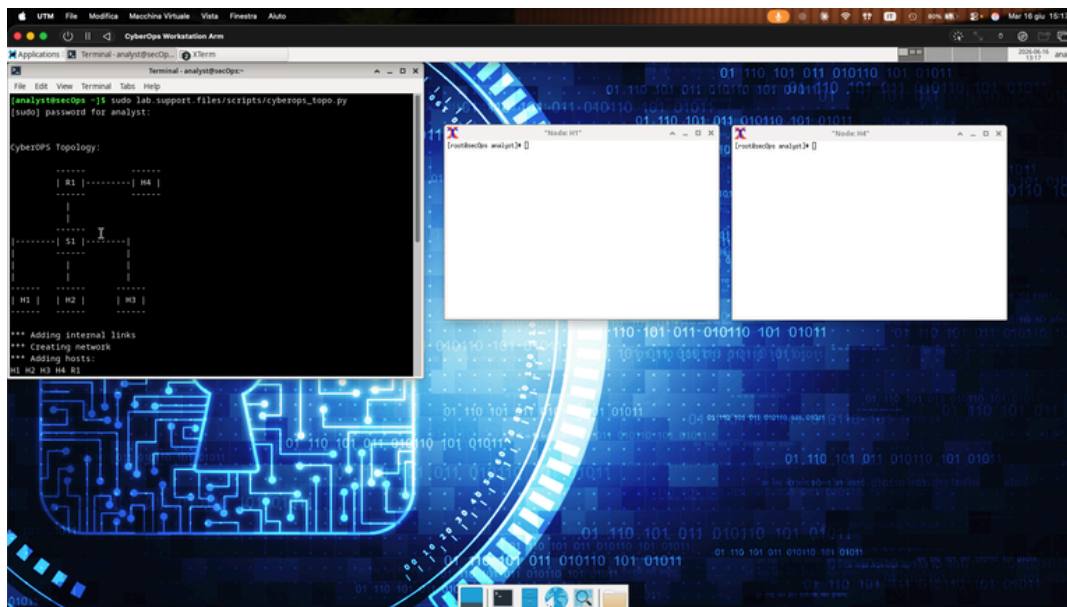
Topologia Mininet: H1 (10.0.0.11) e H4 (172.16.0.40) collegati tramite switch S1 e router R1. H4 funge da server web nginx; H1 da client. Le due macchine si trovano su subnet diverse (10.0.0.0/24 e 172.16.0.0/12), il traffico transita quindi per R1.

2. Parte 1 - Configurazione della topologia e cattura con tcpdump

Step 1 Avvio della topologia Mininet

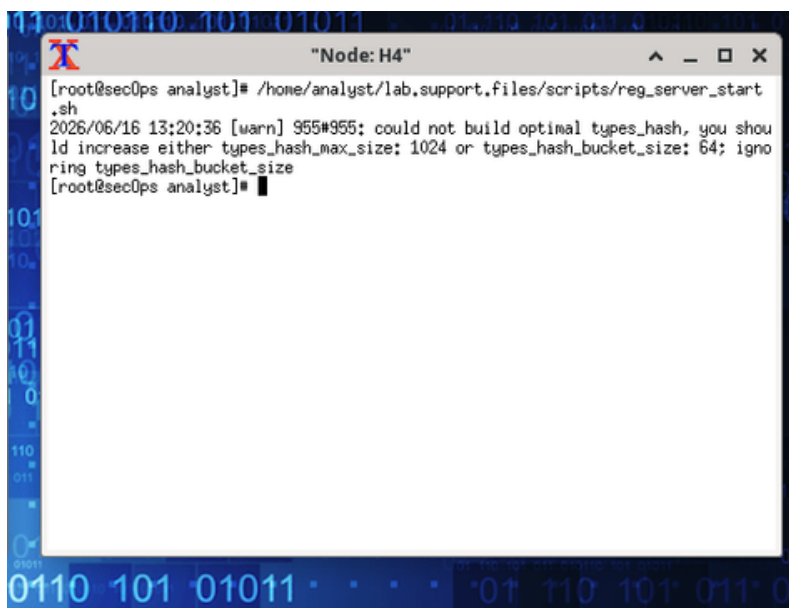
Dal terminale della CyberOps Workstation VM, è stato eseguito lo script di setup con privilegi di root per avviare la topologia Mininet predefinita del corso. Lo script crea la rete virtuale con host, switch e router, configura le tabelle di routing e avvia la CLI interattiva di Mininet.





Step 2 Avvio del server web nginx su H4

Nel terminale del nodo H4 e' stato avviato il server web nginx tramite lo script `reg_server_start.sh`. Lo script configura nginx come server HTTP sulla porta 80 e lo mette in ascolto sull'indirizzo 172.16.0.40. Il warning relativo a `types_hash_max_size` e' atteso e non impatta il funzionamento del server.

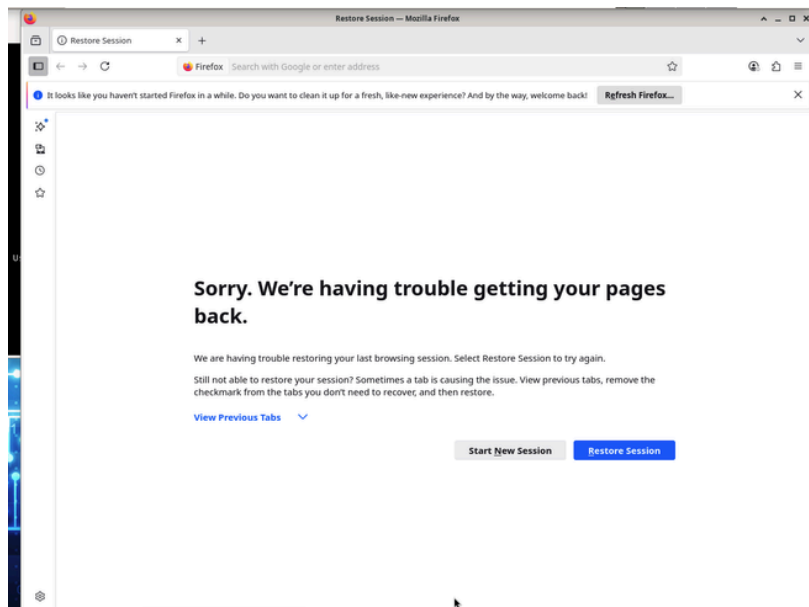


Step 3 Avvio di Firefox e cattura tcpdump su H1

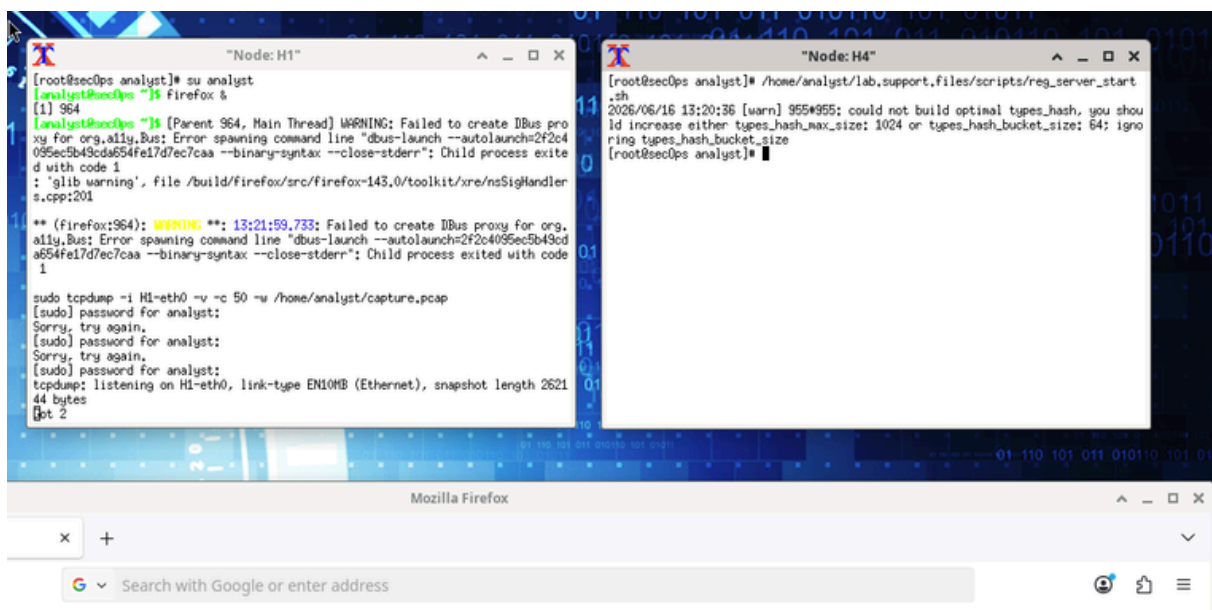
Nel terminale del nodo H1, e' stato eseguito prima su `analyst` per operare come utente standard, poi `firefox &` per avviare il browser in background. Contemporaneamente, da un secondo terminale con privilegi root su H1, e' stato avviato `tcpdump` con il comando:

```
sudo tcpdump -i H1-eth0 -v -c 50 -w /home/analyst/e.pcap
```

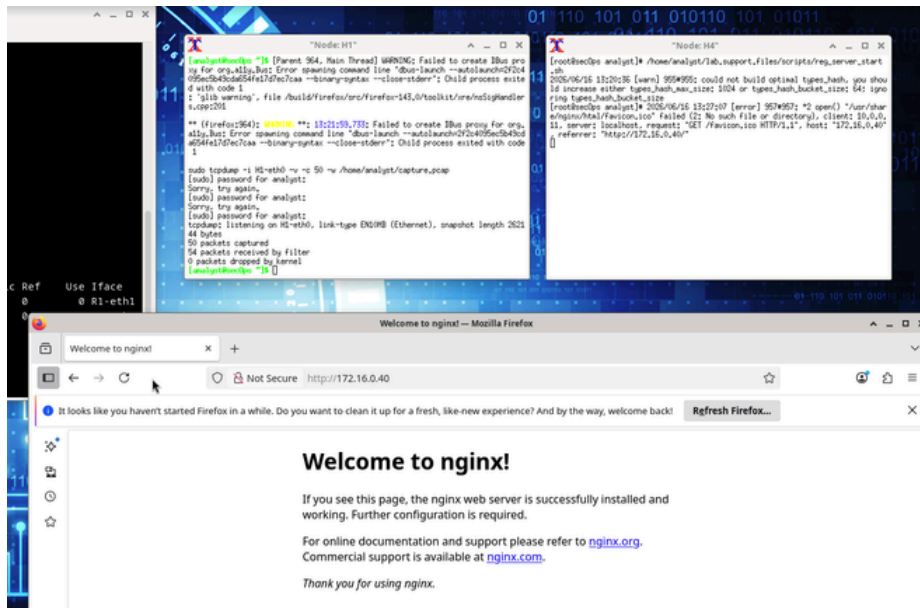
I parametri utilizzati: `-i H1-eth0` specifica l'interfaccia di cattura, `-v` abilita l'output verboso, `-c 50` limita la cattura a 50 pacchetti, `-w` salva i pacchetti nel file binario `e.pcap` invece di mostrarli a video.



3. Parte 2 - Analisi dell'handshake TCP con Wireshark



Nel browser e' stato inserito l'indirizzo `http://172.16.0.40` (IP di H4). Firefox ha caricato correttamente la pagina di benvenuto di nginx. Subito dopo, tcpdump ha raggiunto il limite di 50 pacchetti e ha concluso la cattura, salvando il file `e.pcap`.



D: Quanti pacchetti sono stati catturati e in quale file?

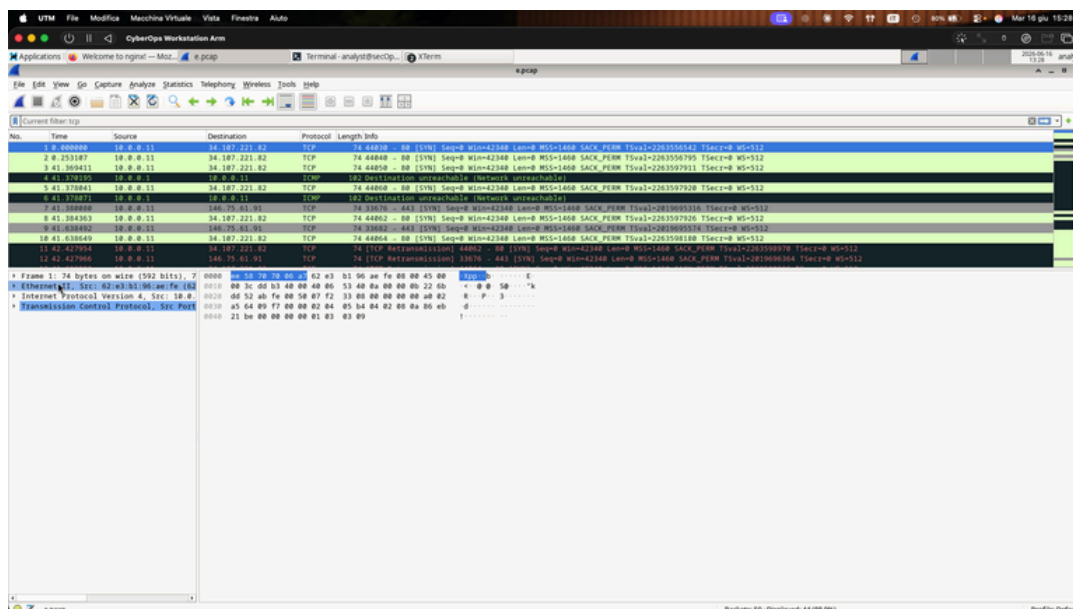
R: Sono stati catturati esattamente 50 pacchetti (limite imposto con -c 50) e salvati nel file binario /home/analyst/e.pcap. Nota: le istruzioni del laboratorio suggerivano il nome capture.pcap, ma il file e' stato nominato e.pcap. Questo ha causato un errore al passo di lettura successivo, risolto utilizzando il nome corretto.

3. Parte 2 - Analisi dell'handshake TCP con Wireshark

Il file e.pcap e' stato aperto in Wireshark (avviato con wireshark-gtk e.pcap dal terminale della VM). Wireshark ha caricato tutti i 50 pacchetti catturati.

Step 4 Applicazione del filtro e individuazione dei frame dell'handshake

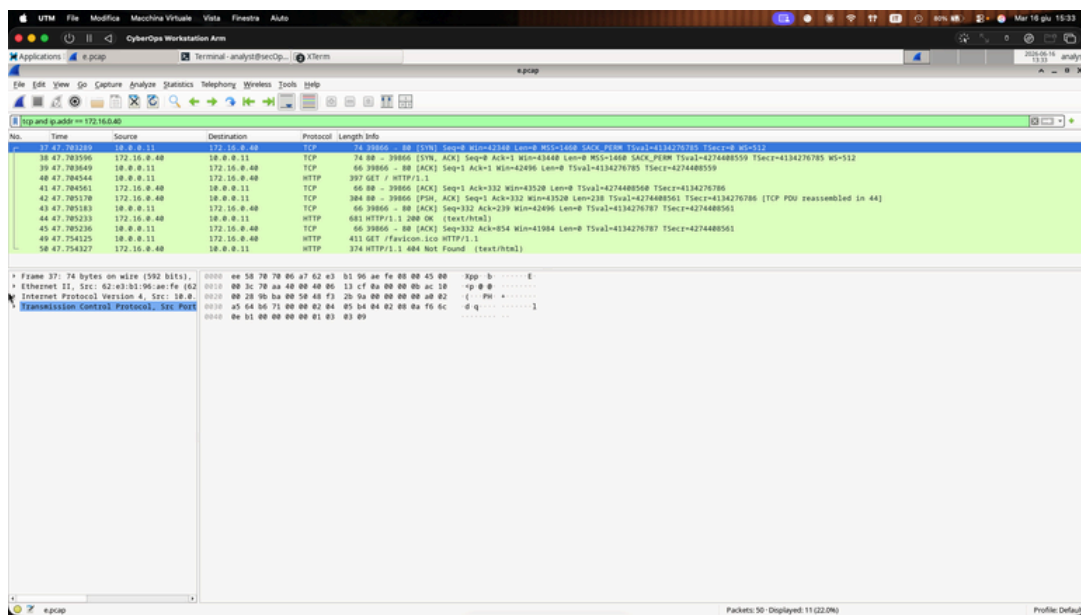
Applicando inizialmente il filtro generico tcp, i primi pacchetti visualizzati erano connessioni TCP verso indirizzi IP di Google (es. 82.221.107.34), non verso H4. Questo era dovuto alle connessioni di background di Firefox verso server esterni (telemetria, aggiornamenti).



D: Perché il filtro tcp mostrava traffico verso Google invece che verso H4?

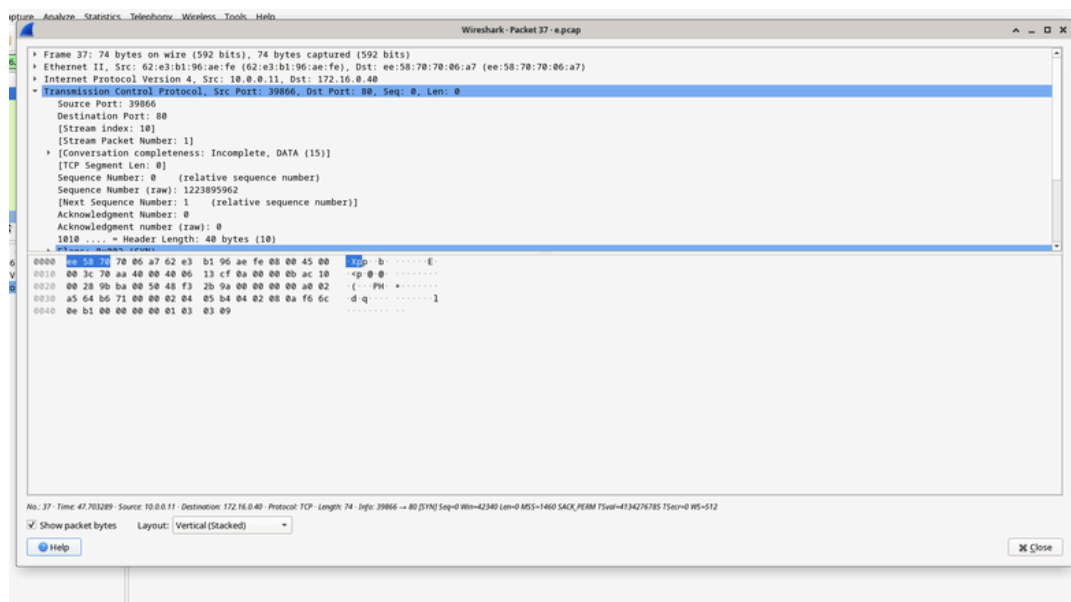
R: Firefox apre automaticamente connessioni TCP verso server esterni anche senza interazione dell'utente (telemetria, controllo aggiornamenti, Safe Browsing). Il filtro generico tcp catturava tutto il traffico TCP sull'interfaccia, incluso questo traffico di background. Per isolare solo i pacchetti relativi alla connessione verso H4 e' stato necessario specificare l'indirizzo IP con il filtro composto: `tcp and ip.addr == 172.16.0.40`.

Applicando il filtro `tcp and ip.addr == 172.16.0.40`, Wireshark ha isolato i pacchetti relativi esclusivamente alla connessione tra H1 (10.0.0.11) e H4 (172.16.0.40). I tre frame dell'handshake sono risultati essere i frame 37, 38 e 39 del file di cattura.



Step 5 Frame 37 - SYN (richiesta di connessione da H1)

Il Frame 37 rappresenta il primo passo dell'handshake: H1 (10.0.0.11) invia a H4 (172.16.0.40) un segmento TCP con il flag **SYN** attivo, comunicando la propria volonta' di aprire una connessione e il numero di sequenza iniziale.



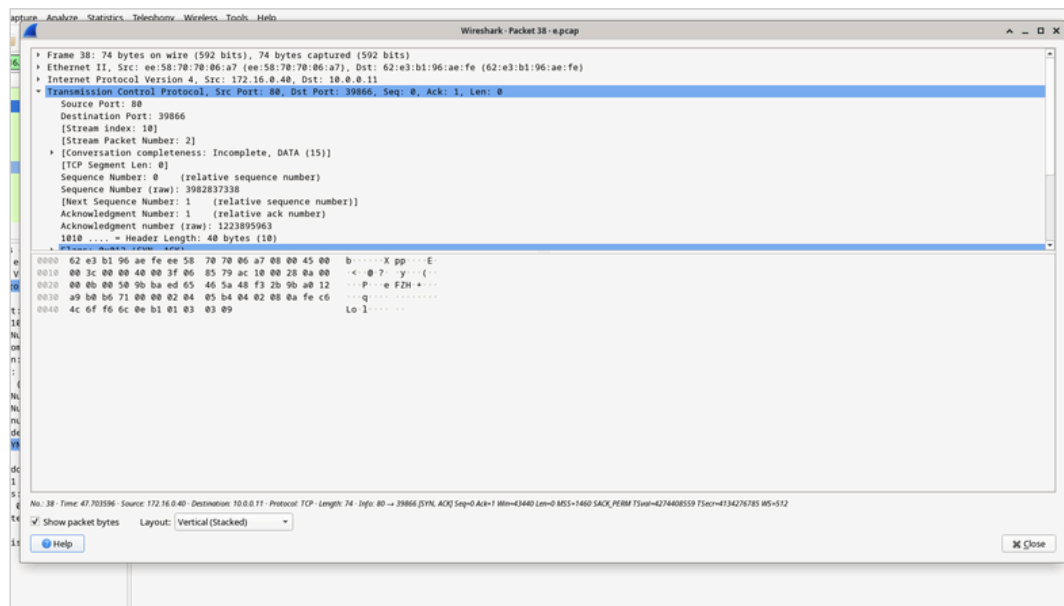
Campo	Valore	Significato
Source Port	39866	Porta efimera del client H1 (assegnata dinamicamente dal kernel)
Destination Port	80	Porta well-known HTTP del server H4 nginx
Sequence Number	0 (relativo)	Numero di sequenza iniziale (ISN) di H1 - valore raw: 1223895962
Ack Number	0	Non significativo nel SYN, connessione non ancora stabilita
Flags	0x002 (SYN)	Solo il flag SYN e' attivo: richiesta di apertura connessione

D: Qual e' la differenza fra la porta sorgente 39866 e la porta destinazione 80?

R: La porta **80** e' una porta well-known: assegnata permanentemente al protocollo HTTP da IANA (range 0-1023), il server e' sempre raggiungibile su quell'indirizzo fisso. La porta **39866** e' una porta efimera (o dinamica): assegnata temporaneamente dal sistema operativo al client per la durata della singola connessione (range Linux tipico 32768-60999). Al termine della sessione, la porta efimera viene liberata e potra' essere riutilizzata.

Step 6 Frame 38 - SYN-ACK (risposta del server H4)

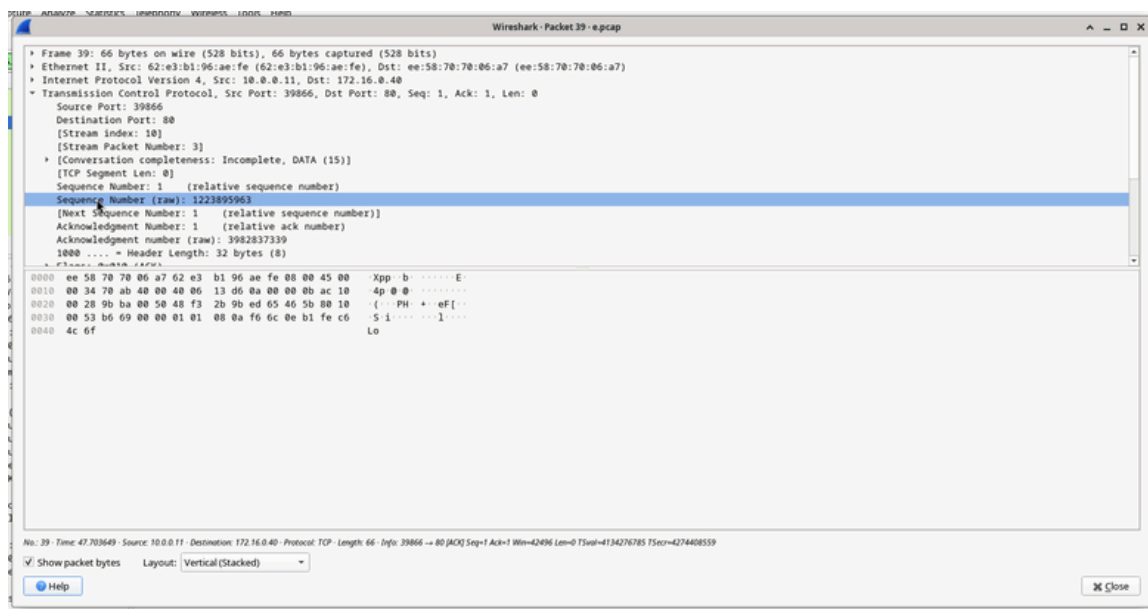
Il Frame 38 e' la risposta di H4 al SYN ricevuto: il server conferma la ricezione (flag **ACK**) e comunica il proprio numero di sequenza iniziale (flag **SYN**), completando il secondo passo dell'handshake.



Campo	Valore	Significato
Source Port	80	Il server H4 risponde dalla porta HTTP
Destination Port	39866	Risponde alla porta efimera del client H1
Sequence Number	0 (relativo)	ISN del server H4 - valore raw: 3982837338
Ack Number	1 (relativo)	Conferma di aver ricevuto il SYN di H1 (ISN+1)
Flags	0x012 (SYN+ACK)	Entrambi SYN e ACK attivi: accettazione e sync simultanei

Step 7 Frame 39 - ACK (conferma finale da H1)

Il Frame 39 chiude l'handshake: H1 invia ad H4 un segmento con il solo flag **ACK**, confermando la ricezione del SYN-ACK del server. Dopo questo frame la connessione TCP e' stabilita e H1 puo' inviare la richiesta HTTP GET.



Campo	Valore	Significato
Sequence Number	1 (relativo)	ISN di H1 + 1 (incrementato dopo il SYN)
Ack Number	1 (relativo)	Conferma di aver ricevuto il SYN di H4 (ISN server +1)
Flags	0x010 (ACK)	Solo ACK: nessun dato, pura conferma - connessione stabilita

Riepilogo handshake: Frame 37: H1 invia SYN (Seq=0) | Frame 38: H4 risponde SYN-ACK (Seq=0, Ack=1) | Frame 39: H1 conferma ACK (Seq=1, Ack=1) - connessione TCP stabilita in 3 passi.

4. Parte 3 - Lettura del pcap con tcpdump e pulizia Mininet

Step 8 Opzione -r di tcpdump (man tcpdump)

Prima di rileggere il file, e' stato consultato il manuale di tcpdump per verificare il significato dell'opzione -r:

```
man tcpdump

-r file
Read packets from file (which was created with the -w option or by
other tools that write pcap or pcapng files). Standard input is used
if file is '-'.
```

D: Cosa fa l'opzione -r di tcpdump?

R: L'opzione -r indica a tcpdump di **leggere i pacchetti da un file** .pcap gia' salvato, invece di catturarli live da un'interfaccia di rete. E' il complementare dell'opzione -w (che scrive) e permette di analizzare la cattura in un secondo momento, filtrare i pacchetti, o trasferire il file su un'altra macchina per l'analisi.

Step 9 Lettura del file e.pcap con tcpdump -r

Dal nodo H1 e' stato eseguito il comando per leggere i primi 3 pacchetti TCP del file di cattura. Il primo tentativo con il nome sbagliato (capture.pcap) ha restituito un errore; il secondo con il nome corretto ha funzionato:

```
tcpdump -r /home/analyst/capture.pcap tcp -c 3
tcpdump: /home/analyst/capture.pcap: No such file or directory

tcpdump -r /home/analyst/e.pcap tcp -c 3
reading from file /home/analyst/e.pcap, link-type EN10MB (Ethernet)...
13:26:19 IP 10.0.0.11.44030 > 82.221.107.34.bc.googleusercontent.com.http
13:26:19 IP 10.0.0.11.44040 > 82.221.107.34.bc.googleusercontent.com.http
13:27:00 IP 10.0.0.11.44050 > 82.221.107.34.bc.googleusercontent.com.http
```



```
"Node: H1"
[root@secOps analyst]# tcpdump -r /home/analyst/capture.pcap tcp -c 3
tcpdump: /home/analyst/capture.pcap: No such file or directory
[root@secOps analyst]# tcpdump -r /home/analyst/e.pcap tcp -c 3
reading from file /home/analyst/e.pcap, link-type EN10MB (Ethernet), snapshot le
ngth 262144
13:26:19.526998 IP 10.0.0.11.44030 > 82.221.107.34.bc.googleusercontent.com,http
: Flags [S], seq 133313288, win 42340, options [mss 1460,sackOK,TS val 226355654
2 ecr 0,nop,wscale 9], length 0
13:26:19.780105 IP 10.0.0.11.44040 > 82.221.107.34.bc.googleusercontent.com,http
: Flags [S], seq 2275024858, win 42340, options [mss 1460,sackOK,TS val 22635567
95 ecr 0,nop,wscale 9], length 0
13:27:00.896409 IP 10.0.0.11.44050 > 82.221.107.34.bc.googleusercontent.com,http
: Flags [S], seq 1666320953, win 42340, options [mss 1460,sackOK,TS val 22635979
11 ecr 0,nop,wscale 9], length 0
[root@secOps analyst]#
```

D: Perché i 3 pacchetti mostrati da tcpdump -r sono verso Google e non verso H4?

R: Il file e.pcap contiene 50 pacchetti in ordine cronologico. I primi pacchetti TCP catturati erano le connessioni di background di Firefox verso server Google (avviate automaticamente all'apertura del browser), che hanno preceduto temporalmente la navigazione manuale verso 172.16.0.40. Il flag -c 3 mostra solo i primi 3, tutti di Firefox. Per vedere i frame dell'handshake H4 sarebbe necessario usare il filtro: `tcpdump -r e.pcap 'host 172.16.0.40' -c 3`.

Step 10 Chiusura di Mininet e pulizia

Al termine del laboratorio, dalla CLI di Mininet è stato digitato quit per arrestare tutti i nodi virtuali, gli switch, i link e il controller. Successivamente, per ripulire completamente eventuali residui (interfacce, datapaths OVS, processi stale), è stato eseguito il comando di cleanup:

```
sudo mn -c
```

```
Terminal - analyst@secOps:~
File Edit View Terminal Tabs Help
s1 ...
*** Routing Table on Router:
Kernel IP routing table
Destination Gateway Genmask Flags Metric Ref Use Iface
10.0.0.0 0.0.0.0 255.255.255.0 U 0 0 0 R1-eth1
172.16.0.0 0.0.0.0 255.240.0.0 U 0 0 0 R1-eth2

*** Starting CLI:
mininet> xterm H1
mininet> xterm H4
mininet> xterm H1
mininet> quit
*** Stopping 0 controllers

*** Stopping 3 terms
*** Stopping 5 links
....
*** Stopping 1 switches
s1
*** Stopping 5 hosts
H1 H2 H3 H4 R1 Disabling IP forwarding on R1

*** Done
[analyst@secOps ~]$
```

```
[analyst@secOps ~]$ sudo mn -c
[sudo] password for analyst:
*** Removing excess controllers/ofprotocols/ofdatapaths/pings/noxes
killall controller ofprotocol ofdatapath ping nox_core lt-nox_core ovs-openflowd ovs-controller ovs-testco
ntroller udpbwtest mnexec ivs ryu-manager 2> /dev/null
killall -9 controller ofprotocol ofdatapath ping nox_core lt-nox_core ovs-openflowd ovs-controller ovs-tes
tcontroller udpbwtest mnexec ivs ryu-manager 2> /dev/null
pkill -9 -f "sudo mnexec"
*** Removing junk from /tmp
rm -f /tmp/vconn* /tmp/vlogs* /tmp/*.out /tmp/*.log
*** Removing old X11 tunnels
*** Removing excess kernel datapaths
ps ax | egrep -o 'dp[0-9]+' | sed 's/dp/nl:/'
egrep: warning: egrep is obsolescent; using grep -E
*** Removing OVS datapaths
ovs-vsctl --timeout=1 list-br
ovs-vsctl --timeout=1 list-br
*** Removing all links of the pattern foo-ethX
ip link show | egrep -o '([[:alnum:]]+-eth[[:digit:]]+)'
egrep: warning: egrep is obsolescent; using grep -E
ip link show
*** Killing stale mininet node processes
pkill -9 -f mininet:
*** Shutting down stale tunnels
pkill -9 -f Tunnel=Ethernet
pkill -9 -f .ssh/mn
rm -f ~/.ssh/mn/*
*** Cleanup complete.
[analyst@secOps ~]$
```

5. Riflessioni finali

D: Indicare almeno 3 filtri Wireshark utili per analisi di rete (diversi da 'tcp').

R: 1. **http** - Mostra solo il traffico HTTP (GET, POST, risposte 200/404 ecc.), utile per analizzare il contenuto delle richieste web in chiaro. 2. **dns** - Filtra le query e risposte DNS, fondamentale per verificare la risoluzione dei nomi e individuare exfiltration via DNS tunneling. 3. **icmp** - Visualizza solo i ping (Echo Request/Reply), utile per diagnosticare raggiungibilit  e latenza o individuare port scan con ping sweep. 4. **ip.src == 10.0.0.11** - Mostra solo il traffico originato da uno specifico host, utile per isolare il comportamento di una singola macchina.

D: In quali altri scenari reali e' utile Wireshark o un packet analyzer?

R: In ambito professionale, i packet analyzer vengono usati per: **troubleshooting di rete** (diagnosi di latenze, pacchetti persi, connessioni che non si stabiliscono); **analisi di malware** (osservare le connessioni C2 di un campione in sandbox, identificare protocolli e IP di comando e controllo); **audit di sicurezza** (verificare che il traffico sia effettivamente cifrato, rilevare comunicazioni in chiaro con credenziali); **incident response** (ricostruzione della timeline di un attacco da pcap registrati da IDS/IPS o da tap di rete); **sviluppo e test di protocolli** (verifica del corretto funzionamento di un'implementazione custom).

6. Conclusioni

Questo laboratorio ha permesso di osservare concretamente il TCP Three-Way Handshake a livello di singolo frame, vedendo i flag SYN, SYN-ACK e ACK, i numeri di sequenza e i numeri di acknowledge propagarsi tra i due host. La comprensione di questi meccanismi e' fondamentale in Cyber Security: anomalie nell'handshake (es. SYN flood, half-open connections, RST injection) sono alla base di numerosi attacchi TCP e tecniche di evasione IDS.

L'esperimento ha evidenziato anche un punto pratico importante: l'uso di filtri Wireshark precisi e' indispensabile in ambienti con traffico misto. Il filtro generico tcp mostrava traffico di background di Firefox, rendendo difficile isolare i frame di interesse. Il filtro composto `tcp and ip.addr == 172.16.0.40` ha isolato immediatamente i tre frame dell'handshake. La stessa logica si applica nelle analisi reali su reti di produzione, dove il volume di traffico rende indispensabile un filtraggio preciso.